Unification

Date .....

UNIFY(knows(John,x), knows(y,z)) could return {y/John, x/z} or {y/John, x/John, z/John}

- the 1st unifier gives knows(John,z) as the result of unification.
- The 2nd unifier gives knows(John, John)
- The 2nd result is obtained by the additional substitution of {z/John}
- By this the first unifier is more general than the second.
b/c it places fewer restrictions on the values of the variables.
It is most general unifier (MGU)

storage and retrieval:

Tell and Ask functions used to inform and interrogate a KB

- These are most primitive functions.

different
y with N1

- Remove ¬missile(y) and missile(M1)

STORE(s) stores a sentence into the KB and FETCH(q) returns all unifiers such that the query q unifies with some sentence in the KB.

- The implement STORE and FETCH is to keep all the facts in the KB in one long list, and given a query q, call UNIFY(q, s) for every sentence s in the list.

Lifting refers to transforming a logical representation from a First order form (FOF) (grounded, concrete) to a higher order form (HOF) (abstract, symbolic)

- This process enables AI systems to handle more complex reasoning tasks by working with variables and constraints instead of just specific instances.

- Lifting is essential about making algorithms and reasoning more general and reusable.

Date .....

Resolution:

Resolution is a theorem proving technique that proceeds by building refutation proofs. i.e proof by contradictions.

- Resolution was invented by John Alan Robinson in the year 1965.
- Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements.
- Resolution is a single inference rule which can efficiently operate on the CNF or clausal form

There are mainly four steps.

- Conversion of facts into FOL
- Convert FOL statements into CNF (Conjunctive Normal form)
- Negate the statement which need to prove (proof by contradiction)

• Draw resolution graph (unification)

st1: It is a crime for an American to sell weapons to hostile nations.

1. $\forall x, y, z$ American$(x) \wedge$ weapon$(y) \wedge$ sells$(x, y, z) \wedge$ hostile$(z) \Rightarrow$ criminal(x)

st2. Nono has some missiles

Some means existential quantifier we will use and as the connection between two colours.

st3: i.e $\exists x$ owns$(Nono, x) \wedge$ missile(x):

owns$(Nono, M_1)$ and

Rule 2

Rule 3 missile(M_1)

from st2 we will get 2 rules R_2 and R_3.

st4: all of its missiles were sold to it by colonel west

Rule4: $\forall x$ missile$(x) \wedge$ owns$(Nono, x) \Rightarrow$ sells$(west, x, Nono)$

An enemy of America counts as hostile.

Date .....

R_5 : $\forall x$ Enemy $(x, \text{America}) \Rightarrow$ Hostile (x)

West who is American

R_6 : American (West)

The country Nono, an ~~enemy~~ enemy of America

R_7 : Enemy (Nono, America)

All missiles are weapons

R_8 : $\forall x$ missile $(x) \Rightarrow$ weapon (x)

$\alpha \rightarrow \beta = \neg\alpha \vee \beta$

$\neg$ American $(x) \vee \neg$ weapon $(y) \vee \neg$ Sells $(x, y, z) \vee$
$\neg$ Hostile $(z) \vee$ criminal (x)

- owns (Nono, M_1)
- Missile (M_1)
- $\neg$ Missile $(x) \vee \neg$ own (Nono, x) $\vee$ sells (West, x, Nono)
- $\neg$ enemy $(x,$ America$) \vee$ Hostile (x)
- American (West)

different ~~no~~ "11"

- Enemy(Nono, America)

from the given data we have to prove criminal west.

To prove that we have to negate statement. $\boxed{\neg \text{criminal(west)}}$

To consider the resolution we have to find contradictary statement i.e criminal(west)

$\boxed{\neg \text{American}(x) \vee \neg \text{weapon}(y) \vee \neg \text{sells}(x, y, z) \vee \text{criminal}(x)}$

$\neg \text{criminal(west)}$

performing unification between two statements. where there is x replaced by west

$\boxed{\neg \text{American(west)} \vee \neg \text{weapon}(y) \vee \neg \text{sells(west}, y, z)}$

and no need to consider criminal(x) and $\neg$ criminal(west).

find the clause there exist with the contradiction i.e American(west)

- applying unification b/w 2 statements ¬American(west) and American(west) can be removed

- The remaining is ~~¬weapon(y) v ¬sells(west~~ ¬weapon(y) v ¬sells(west,y,z) v ¬Hostile(z)

- consider the clause that is in contradiction.

so ¬missile(x) v weapon(x)

performing unification so weapon(x) and ¬weapon(y) can be removed.

so the remaining statement is

¬missile(y) v ¬sells(west,y,z) v ¬Hostile(z)

in missile(x) x is replaced with y

- consider the clause which is contradiction ie missile(M1) the variable are different do apply unification so replace y with M1

- Remove ¬missile(y) and missile(M1)

so the remaining statement is

[¬ sells(x, M1, z) ∨ ¬ Hostile(z)]

considering the clause to implement contradiction ie

[¬ missile(x) ∨ ¬ owns(Nono, x) ∨ sells(West, x, Nono)]

To implement contradiction the variables are not same ind sells so replace. the constants 'x' with 'M1' and 'z' with 'Nono'

The statement looks like

[¬ missile(M1) ∨ ¬ owns(Nono, M1) ∨ ¬ Hostile(Nono)]

conside another clause.

ie [missile(M1)]

so remove missile(M1) and ¬missile(M1)

Remaining

[¬ owns(Nono, M1) ∨ ¬ Hostile(Nono)]

consider the clause to implemend contradiction, so [owns(Nono, M1)]

Remove owns(Nono, M1) and negation left over [¬ Hostile(Nono)]

- considering the clause to implement contradiction ¬Enemy(x, America) ∨ Hostile(x)

replace the constant 'x' with Nono

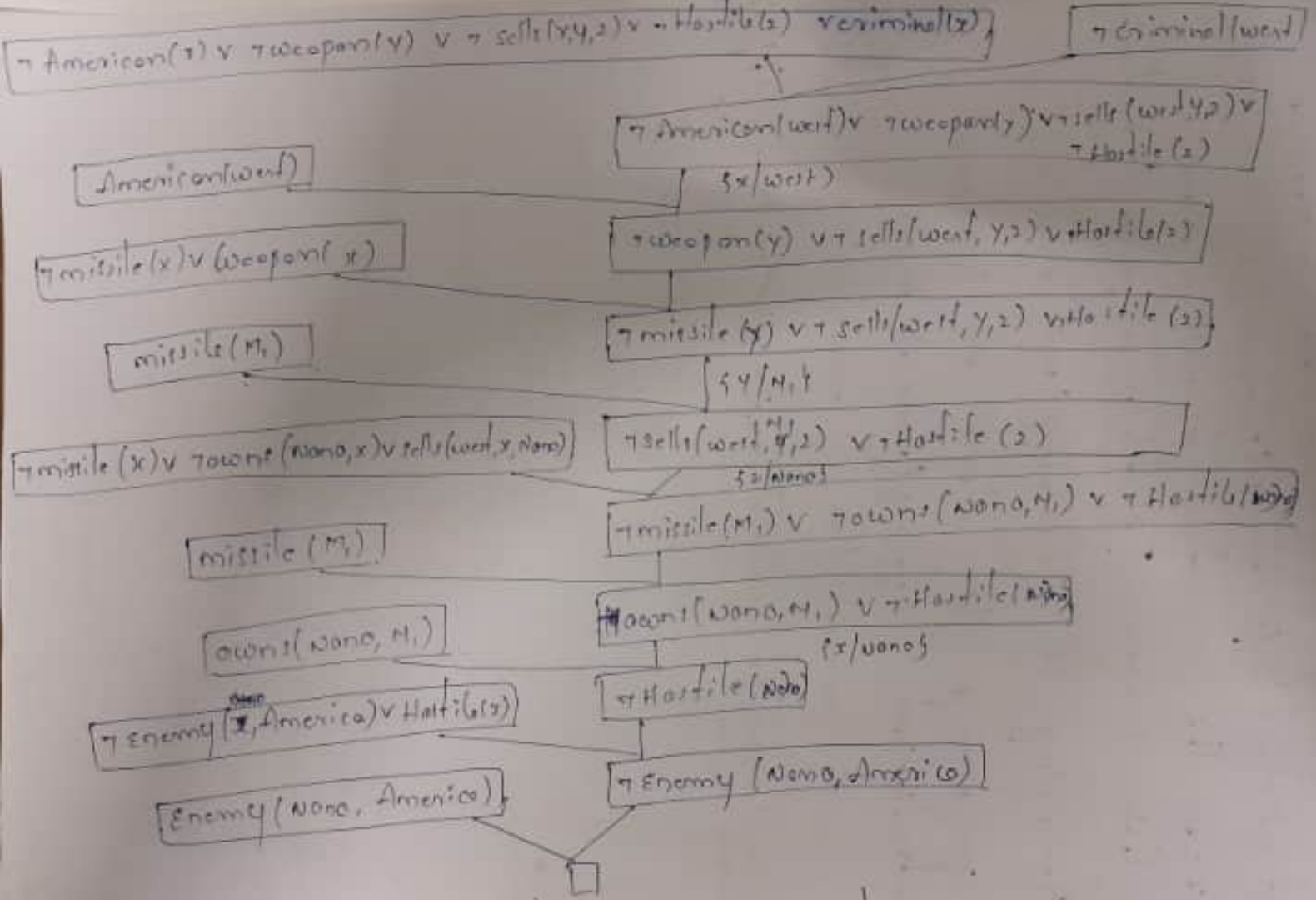
then remove ¬Hostile(Nono) and ¬Hostile(Nono)

- Remaining is ¬Enemy(Nono, America)

- considering the clause to implement contradiction Enemy(Nono, America)

- so remove the enemy.

¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x,y,z) ∨ ¬Hostile(z) ∨ Criminal(x)


¬ American(x) ∨ ¬ weapon(y) ∨ ¬ sells(x,y,z) ∨ ¬ Hostile(z) ∨ criminal(x)
¬ criminal(west)
¬ American(west) ∨ ¬ weapon(y) ∨ ¬ sells(west,y,z) ∨ ¬ Hostile(z)
American(west)
{x/west}
¬ missile(x) ∨ weapon(x)
¬ weapon(y) ∨ ¬ sells(west,y,z) ∨ ¬ Hostile(z)
missile(M1)
¬ missile(y) ∨ ¬ sells(west,y,z) ∨ ¬ Hostile(z)
{y/M1}
¬ missile(x) ∨ ¬ owns(Nono,x) ∨ sells(west,x,Nono)
¬ sells(west,M1,z) ∨ ¬ Hostile(z)
{z/Nono}
missile(M1)
¬ missile(M1) ∨ ¬ owns(Nono,M1) ∨ ¬ Hostile(Nono)
owns(Nono,M1)
¬ owns(Nono,M1) ∨ ¬ Hostile(Nono)
{x/Nono}
¬ Enemy(x,America) ∨ Hostile(x)
¬ Hostile(Nono)
Enemy(Nono,America)
¬ Enemy(Nono,America)
□
A resolution proof that is criminal

Using first order logic

Assertions and queries in FOL:

Sentences are added to KB using TELL, exactly as in PL. Such sentences are called assertions.

- we can assert that John is a king and that kings are persons.

TELL (KB, king(John))

TELL (KB, $\forall x$ king(x) => person(x))

- we can ask query by using ASK

ASK (KB, king(John))

returns True, b/c John is a king exists in the KB.

Ask are called queries or goals.

Goals are used to describe an agent's desired states.

- Any query is entailed with knowledgebase
- For this following query it returns true.

ASK (KB, person(John))

- we can write it as quantified query as

ASK (KB, $\exists x$ person(x))

It will result true based on {x/John}.

if there is more than one possible answer the list of substitutions can be returned.

Kinship domain
numbers, sets and lists
. wumpus world.

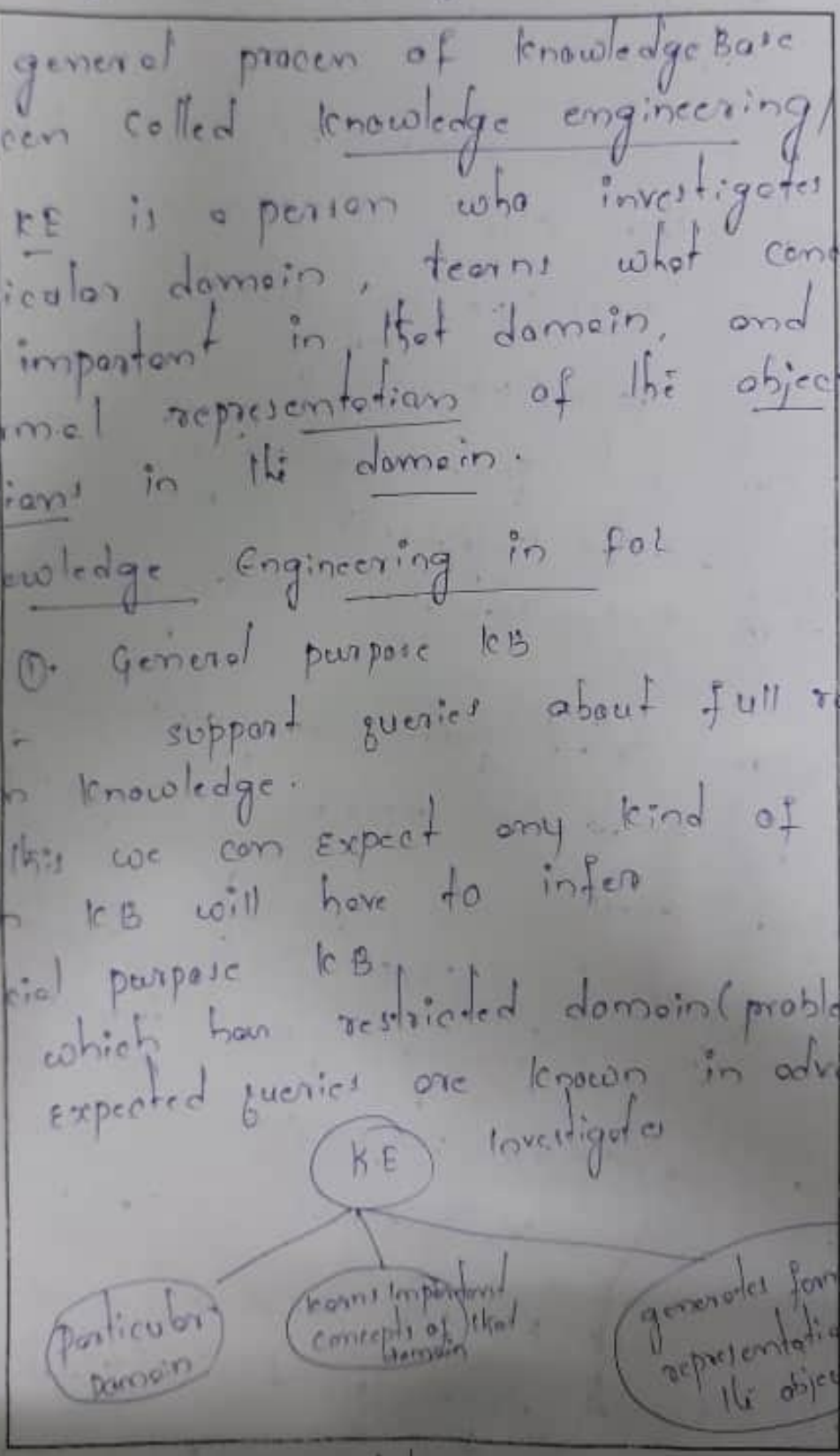
Knowledge Engineering in fol:

The general process of knowledge Base construction a process called knowledge engineering / k.Engineer

- A KE is a person who investigates a particular domain, learns what concepts are important in that domain, and creates a formal representation of the objects and relations in the domain.

- Knowledge Engineering in fol

1. General purpose KB
   - support queries about full range of human knowledge.
   - In this case can expect any kind of query, which KB will have to infer
2. special purpose KB.
   which has restricted domain (problem specific) where expected queries are known in advance.



he is called

Name :
Year & Sem
Subject
H.T. No
Date
Signature of Invigilator

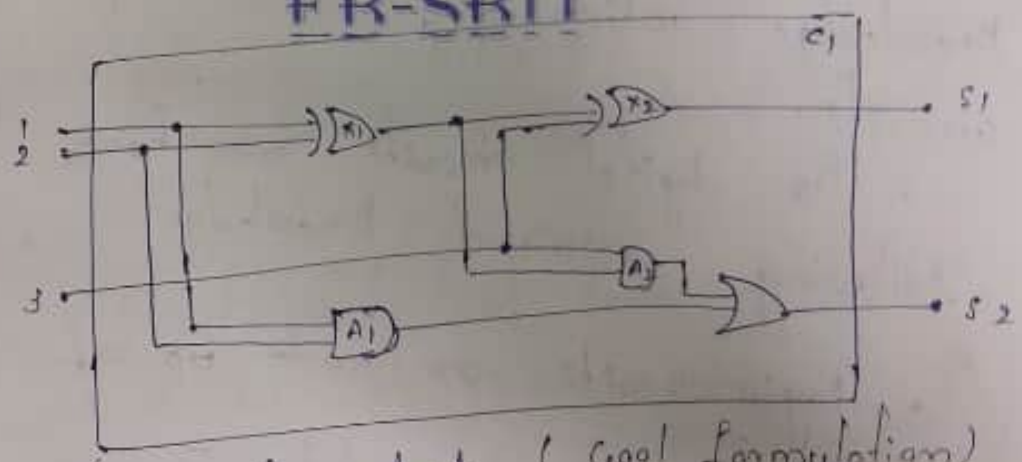
Knowledge - engineering procen:

Following are some main steps of the knowledge - engineering procen.

- using these steps we will develop a KB which will allow us to reason about digital circuit (one-bit fulladder)

[Domain of electronic circuit]

FR-SRIT



1. Identify the task (Goal formulation)
2. Assemble the relevant knowledge (knowledge Acquisition) Acquiring
3. Decide on vocabulary (knowledge representation)
4. Encode general knowledge about the domain (vocabulary terms)
5. Encode a description of the problem instance. (writing simple atomic sentences of instances of concepts)

6. pose queries to the inference engine and get answers (Test the knowledge base with problem instances)

7. Debug the knowledge base. (will try to debug the issues in KB)

1. - Re 1st step of the procen it to identify the task

- for the digital circuit there are various reasoning tasks

- At the first level or highest level we will examine the functionality of the circuit.

- we will examine circuit's structure.

(2) - In this we will assemble the relevent knowledge which is required for digital circuits.

- To digital circuits we have the following required knowledge

- logic circuits are made up of wires and gates.

- signal flows through wires to the input terminal of the gate

Each gate produces the corresponding output.

- In logic circuit there are four types of gates used: AND, OR, XOR and NOT.

- All these gate have 2 input terminal and one output terminal.
- only Not gate having one input terminal.

(3) The next step of the process is to select functions, predicates and constants to represent the circuits, terminals, signals and gates.

- we will distinguish the gates from each other and from other objects.

FR-SBIT

- Each gate is represented as an object which is named by a constant such as Gate(x_1).
- The functionality of each gate is determined by its type.
- which is taken as constants such as AND, OR, NOT & XOR.
- circuits are identified by predicate: circuit(C_1).
- For the terminal we will use predicate Terminal(*).

- next we consider terminals, which are identified by the predicate Terminal(x).
- A gate or circuit can have one or more input terminals and one or more output terminals.
  In(1, x_1) to denote the first input terminal for gate x_1.
- The function Arity(c, i, j) says that circuit function Arity specifies for the overall circuits, has i input and j output terminals.
- The connectivity b/w gates can be represented by a predicate, connected, which takes two terminals as arguments, as in Connected(Out(1, x_1), In(1, x_2))
- Finally we need to know whether a signal is on or off.
- We therefore introduce as objects two signal values, 1 and 0 and a function signal(t) that denotes the signal value for the terminal 't'.
- In the given electronic circuit only 3 Gates are used. XOR, AND, OR

4. Encode general knowledge about the domain

To encode the general knowledge about the logic circuit, we need the following rules.

- If two terminals are connected then they have the same signal.

$\forall t_1, t_2$ Terminal(t_1) $\wedge$ Terminal(t_2) $\wedge$ Connected(t_1, t_2)
$\Rightarrow$ Signal(t_1) = Signal(t_2)

- The signal at every terminal is either 1 or 0.

$\forall t$ Terminal(t) $\Rightarrow$ Signal(t) = 1 $\vee$ Signal(t) = 0

- Connected is commutative

$\forall t_1, t_2$ Connected(t_1, t_2) $\Leftrightarrow$ Connected(t_2, t_1)

- Terminal, signal and connected are different functions.

Any circuit can be defined by using only four gates.

$\forall g$ Gate(g) $\wedge$ K = Type(g) $\Rightarrow$ ~~K = AND $\vee$ K = OR $\vee$ K = XOR $\vee$ K = NOT~~

$\Rightarrow$ K = AND $\vee$ OR $\vee$ XOR $\vee$ NOT

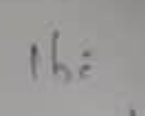
- The behaviour of each gate is given by its type one of the constants AND, OR, XOR, or NOT

Type(x_1) = XOR

- The name of the circuit is represented as Circuit(c_1) only one circuit we are using.

- There are x_1, x_2, A_1, A_2, O_1 Gates the functionality will differ.

Type(x_1) = XOR
Type(x_2) = XOR
Type(A_1) = AND
Type(A_2) = AND
Type(O_1) = OR



⑤ Using the 4th step we will design the encode the specific problem instance.

First we categorize the gate.

Type(x_1) = XOR
Type(x_2) = XOR
Type(A_1) = AND
Type(A_2) = AND
Type(O_1) = OR.

we show the connections between gates

Circuit(c) Arity(c, 3, 2)

we are having circuit c, represents the Arity function of c, with 3 i/ps and 2 o/ps.

- Gate(x_1) ∧ Type(x_1) = XOR
- Gate(x_2) ∧ Type(x_2) = XOR
- Gate(A_1) ∧ Type(A_1) = AND
- Gate(A_2) ∧ Type(A_2) = AND
- Gate(O_1) ∧ Type(O_1) = OR

we show the connections b/w them.

connected(out(1,x_1), In(1,x_2)) connected(In(1,c_1), In(1,x_1))
connected(out(1,x_1), In(2,A_2)) connected(In(1,c_1), In(1,A_1))
connected(out(1,A_2), In(1,O_1)) connected(In(2,c_1), In(2,x_1))
connected(out(1,A_1), In(2,O_1)) connected(In(2,c_1), In(2,A_1))
connected(out(1,x_2), out(1,c_1)) connected(In(3,c_1), In(2,x_2))
connected(out(1,O_1), out(2,c_1)) connected(In(3,c_1), In(1,A_2))

6. pose queries to the inference procedure.
   - we will find whether the KB is working or not.
   what combinations of inputs would cause the first output of c_1 to be 0 and the second output of c_1 to be 1

$\exists i_1, i_2, i_3$ signal(In(1,c_1)) = i_1 ∧ signal(In(2,c_1)) = i_2 ∧ signal(In(3,c_1)) = i_3 ∧ signal(out(1,c_1)) = 0 ∧ signal(out(2,c_1)) = 1

7. Debug

Knowledge Engineering - Benefits

1. Knowledge engineer helps in creating better, smarter, more effective expert system.
2. Models created with knowledge engineering are robust.

Date .....

Converting facts into FOL.

1. John likes all kind of food

$\forall x : food(x) \rightarrow likes(John, x)$

2. Apple and vegetable are food

$food(apple) \wedge food(vegetable)$

3. Anything anyone eats and not killed is food

$\forall x, \forall y : eats(x, y) \wedge \neg killed(x) \rightarrow food(y)$

if x is a person ~~not~~ ~~killed~~ and eats y food and x is not killed is y is food

4. Anil eats peanuts and still alive

~~$\forall x : eats(Anil, x) \rightarrow eats(Harry, x)$~~

$eats(Anil, peanuts) \wedge alive(Anil)$

5. Harry eats everything that Anil eats

$\forall x : eats(Anil, x) \rightarrow eats(Harry, x)$

6. John likes peanuts

$likes(John, peanuts)$

steps for Resolution:

1. conversion of facts into FOL
2. conversion FOL statements into CNF
3. Negate the statement which needs to prove (proof by contradiction)
4. Draw resolution graph (unification).

- cognosis
- vidya
- insights

- A Horn clause is a disjunction of literals.

ex: The clause $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ where $L_{1,1}$ is an agent is +ve literal is a Horn clause

q: $(\neg B_{1,1} \vee P_{1,2} \vee P_2)$, is not.

- Every Horn clause can be written as an implication where premise is a conjunction of +ve literals and where conclusion is a single +ve literal.

EB-SBIT

- The Horn clause $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ can be written an implication $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$

- A definite clause with no negative literals simply asserts a given propositions some-time called fact.

- The positive literal is called head and -ve literals from the body of the clause

- prove the KB $(B \vee D) \wedge C$ Based on the given KB.

$[((A \vee B) \Rightarrow C) \wedge (B \vee \neg D), A \wedge D]$

proof:

$$KB = \{\underbrace{((A \vee B) \Rightarrow C) \wedge (B \vee \neg D)}_{KB_1}, \underbrace{A \wedge D}_{KB_2}\}$$

In this premise two KB are given.

Considering first KB i.e

S_1: $((A \vee B) \Rightarrow C) \wedge (B \vee \neg D)$

apply AND elimination

S_2: $((A \vee B) \Rightarrow C)$

Consider the 2nd KB $A \wedge D$ and apply AND elimination

S_3: $A \wedge D \vdash A$

S_4: A

using a rule called OR Introduction

for S_4 apply OI by using B literal

S_5: $A \vee B$

from statement S_2 and S_5 apply modus ponens $\alpha \rightarrow \beta$ & α Result β

S_6: C

Again from the given KB, consider KB_1 and apply [AND elimination] we get consider known.

S_7: $B \vee \neg D$

from the second KB_2 we get 'D' By applying AND elimination. (AND)

S_8: D

EB-SBIT

($\alpha \vee \neg\beta$, β we get α

a/c to unit resolution

By applying unit resolution on S_6 and S_7 we get.

S_9: B

By applying OR Introduction (OI) we get

S_{10}: $B \vee D$

from S_6 and S_{10} if I introducing AND Introduction it will give

S_{11}: $(B \vee D) \wedge C$

Hence proved.

Drawbacks of P.C.

- propositional logic is very simple and declarative in which knowledge and inference are seperate, and inference is entirely domain independent
- p.c has lack of Datastructure in programming
- p.c is not sufficient for complex or natural language sentences.
  eg: some students in KEC are intelligent
- p.c has very limited expressive power
  eg: cannot say pits cause breezes in adjacent squares" writing one sentence for each square

EB-SBIT

First order Logic (FOL) : FOL is also known as predicate logic or first order predicate logic

- FOL like natural language has well defined syntax & semantics.
- It assumes the world contains
  - objects : people, houses, numbers, colors, shapes ..
  - Relations : red, round, prime, brother ....
  - functions : father of , best friend of ...
- It has ability to represent facts about some or all of the objects & relations in the universe.
- Represent law and rules extracted from real world.
- useful language for Maths, philosophy and AI
- Represent facts in realistic manner rather than just true / false.
- makes ontological commitment (important)
  It is a branch metaphysics (property).
  (mean studying being existence and quality).

Syntax of FOL: Basic elements

- constants : KingJohn, 2, NU1 ..
- predicates : Brother, >, ...
- functions : Sqrt, LeftLegOf, ...
- Variables : x, y, a, b
- connectives : ¬ ∧, ∨, ⇒ ⇔
- equality : =
- quantifiers : ∀, ∃

FR-SRIT

- Sentences : Atomic / complex
- Atom : True/False / P(Atom proposition)
- Complex Sentence : (sentence) / connective sentence
  ¬ sentence.

Quantifiers:

It expresses properties of entire collection of objects, instead of enumerating the objects by name.

FOL contains 2 standard quantifiers.

1. Universal quantifier (∀)
2. Existential " (∃)

Universal quantifier makes statements about every object.

Existential quantifier makes statements about some of the objects.

Atomic sentence = predicate(term$_1$, ... term$_n$)
or term$_1$ = term$_2$

term = function(term$_1$ ... term$_n$) or constant or variable.

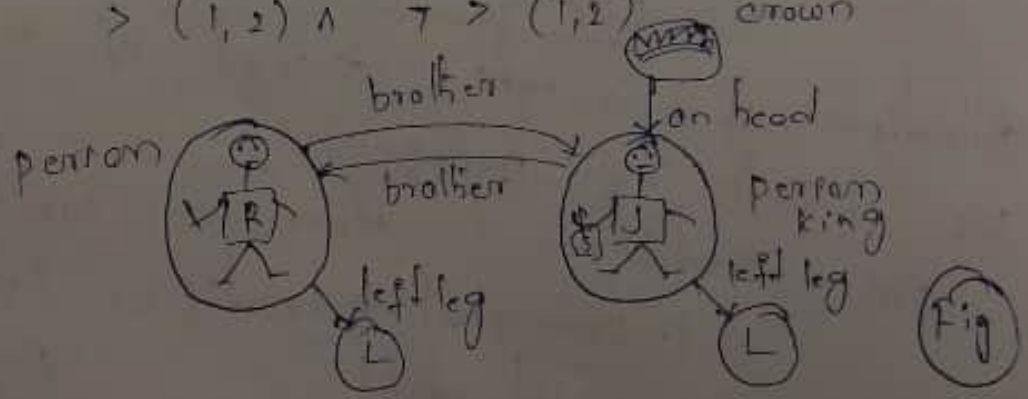
Ex:
- Brother (king John, Richard)
- > (Length (left leg of (Richard)), Length (Left leg of (king John)))

- Complex sentence : There are made from atomic sentences using five logical connectives
- S_1 and S_2 are 2 atomic statements then
- $\neg S_1$, $S_1 \wedge S_2$, $S_1 \vee S_2$, $S_1 \Rightarrow S_2$, $S_1 \Leftrightarrow S_2$

Eg: sibling(king John, Richard) $\Leftrightarrow$ sibling(Richard, king John)

> (1,2) $\vee$ $\leq$ (1,2)

> (1,2) $\wedge$ $\neg$ > (1,2)



Fig

Models for FOL:

Date .....

Models for FOL are interesting, they have objects in them.

- The domain of a model is the set of objects it contains, these objects sometimes called domain elements.

- figure (fig) shows a model with five objects.
  - Richard the lionheart (king) (1189 - 1199)
  - king of England (John) (1199 - 1215) king
  - the left legs of Richard and John
  - king John broken (broken)
  - crown

→ (fig) is a model containing 2 binary relations, 3 unary relations (indicated by labels on objects) and one unary function left leg.

The formal grammar of FOL :- (Backus Naur form)

Sentence → Atomic Sentence
| (Sentence Connective Sentence)
| quantifier variable .. Sentence
| ¬ Sentence

Atomic Sentence → predicate (Term, ...) | Term = Term

Term → function (Term, ...)
| constant
| variable

Connective → ⇒ | ∧ | ∨ | ⇔

quantifier → ∀ | ∃

constant → A | X1 | John | ...

variable → a | x | s | ...

predicate → Before | Hail | Raining | ...

function → Mother | leftLeg | ...

Date .....

Term: A term is a logical expression that refers to an object.
Constant symbols are therefore terms.

Ex: LeftLeg(John)
Interpretation John is a king
John is a person
John is a brother of Richard

Atomic sentence.

Eg:- 1. Brother (Richard, John)
2. Married (Father (Richard), Mother(John))

We have Terms for referring to objects and predicate symbols for referring to relations, we can put together to make atomic sentences.

Complex Sentences:

We use logical connectives to construct more complex sentences.

Ex. winner, person, king &

Ex :- 1. $\neg$ Brother(LeftLeg(Richard), John)

2. Brother(Richard, John) $\wedge$ Brother(John, Richard)

3. King(Richard) $\vee$ King(John)

4. $\neg$ King(Richard) $\Rightarrow$ King(John)

3) Relation
Unary : John is a king
Binary : crown is on head of John
the unary "left leg" function includes the following mapping:
(Richard the lionheart) → Richard's leftleg
(king John) → Johns left leg
symbols and interpretations:

FR-SRIT

symbols are the basic syntactic elements of first order logic
symbols stand for objects, relations & functions
symbols stand for 1. objects
2. Relations
3. functions

symbols are of 3 kinds
1. constant symbols (objects)
ex: John, Richard
2. predicate symbols (Relations)
ex: Onhead, person, king & crown

3. function symbols (functions) ex:- left leg.

- symbols begin with upper case letter.

Models

Truth in FOL

- Sentences are true w.r.t a model and an interpretation.

- Model contains objects (domain elements) and relations among them.

- Interpretation specifies referents for
  - constant symbols - objects
  - predicate " - Relation
  - function " - functional relations with objects

- An atomic sentence predicate(term, ... term$_n$) is true iff the objects referred by term, ... term$_n$ are in the relation referred by predicate.

Models for FOL: Example.

- objects
  - person king John
  - person Richard
  - crown
  - left leg of John
  - left leg of Richard

- Relation:
  - "onhead" < the crown, king John>
  - "brother" < John, Richard>
  - "person" < John>
  - "person" < Richard>
  - "king" <John>
- Function:
  - {no other person wear crown except the king}
  - < John the king> - on head (crown)
  - < John the king> - shoe (left-leg)

FR-SBIT

Semantic of FOL

- An semantic of a fol anigns a notation to all symbols.
- It also determines a domain that specifies the range of the quantifier.
- Each term is assigned an object
- Each predicate is assigned a property of objects,
- Each sentence is assigned a truth value.

Interpretation in FOL:

Domain D be a nonempty set.

- Each constant is assigned an element of D
- Each variable is assigned to subset of D
- Each function 'f' of 'n' is defined on n arguments of D and defines a mapping from D^n into D.

Truth value of FOL

- Expression 'E' and interpretation 'I' for E
  - The value of truth symbols T and F
  - The value of atomic sentence is either T or F, determined by interpretation I
  - The value of constant is the element of D, assigned by I.
  - value of variable is set of elements of D, assigned by I.

Quantifier:

universal quantification: (∀)

- "for all" ∀ <variables> <sentence>
  all the objects in the universe can be considered.

ex:- "All the kings are persons"

- The above sentence written in first order logic as

$\forall x$ king(x) => person(x).

x is a king, x is a person, x is a variable.

- variables can be represented in lowercase.
- A term with no variables is called a ground term.

FR-SBIT

The sentence $\forall x\, p$, where p is any logical expression, says p is true for every object x.

- $\forall x$ p is true in a given model under a given interpretation if p is true in all possible extended interpretation
- By considering the model, the intended interpretation can extend in 5 ways.
  - x → Richard the lionheart
  - x → king John
  - x → Richard's left leg
  - x → "
  - x → The crown.

- A universally quantified sentence

$\forall x$ king(x) $\Rightarrow$ person(x) is true under the original interpretation if sentence king(x) $\Rightarrow$ person(x) is true in each of the five extended interpretation

- universally quantified sentence is equivalent to asserting the following 5 sentences.

Richard the lionheart is a king $\Rightarrow$ Richard the lionheart is a person

King John is a king $\Rightarrow$ King John is a person

Richard's left leg is a king $\Rightarrow$ Richard's left leg is a person

John's left leg is a king $\Rightarrow$ John's left leg is a person

The crown is a king $\Rightarrow$ the crown is a person

Existential quantification ($\exists$)

universal quantification makes statements about every object

- we can make a statement about some objects in the universe without naming it by using an existential quantifier

$\exists x$ Crown(x) $\wedge$ onHead(x, John)

$\exists x$ is pronounced there exists an x

- The sentence $\exists x\ p$ says that p is true for atleast one object x.
- $\exists x\ p$ is true in a given model under a given interpretation if p is true in atleast one extended interpretation that assigns x to a domain element.
- Atleast one of the following must be true:

FR-SBIT

① Richard the lionheart is a crown ∧ Richard the lionheart is on John's head.

② king John is a crown ∧ king John is on John's head

③ Richard's left leg is a crown ∧ Richard left leg is on John's head

④ John's left leg is a crown ∧ John's left leg is on John's head.

⑤ The crown is a crown ∧ the crown is on John's head.

The fifth assertion is true in the model.

- Just as '⇒' appears to be the natural connective to use with '∀'
- '∧' is the natural connective to use with '∃'.
- using '∧' as the main connective with '∀' led to an overly strong statement
- using ⇒ with ∃ usually leads to a very weak statement.

Nested quantifiers:

We often want to express more complex sentences using multiple quantifiers.
- The simplest case is where the quantifiers are of the same type.

Ex:- Brothers are siblings.

$\forall x \forall y$ Brother$(x,y) \Rightarrow$ sibling(x,y)

- Consecutive quantifiers of the same type can be written as one quantifier with several variables. to say that sibling hood is a symmetric relationship we can write

$\forall x,y$ sibling$(x,y) \Leftrightarrow$ sibling(y,x)

Ex: ① Everybody loves somebody" means that for every person, there is someone that person loves. $\forall x \exists y$ Loves(x,y)

"appreciates"

① There is some one who is loved by every one.

$\exists y \forall x \; Loves(x, y)$

Connections between $\forall$ and $\exists$

two quantifiers are actually intimately connected with each other, (a) through negation.

- every one dislikes parsnips is the same as saying there doesn't exist someone who likes them, and viceversa.

$\forall x \; \neg likes(x, parsnips)$ is equivalent to $\neg \exists x \; likes(x, parsnips)$

- every one likes icecream means that there is no one who does not like icecream.

$\forall x \; likes(x, icecream)$ is equivalent to $\neg \exists x \; \neg likes(x, icecream)$.

- Because $\forall$ is really a conjunction over the universe of objects and $\exists$ is a disjunction

- The de Morgan rules for quantified and unquantified sentences are as follows.

$\forall x \neg p \equiv \neg \exists x p$

$\neg \forall x p \equiv \exists x \neg p$

$\forall x p \equiv \neg \exists x \neg p$

$\exists x p \equiv \neg \forall x \neg p$

$\neg p \wedge \neg q \equiv \neg (p \vee q)$

$\neg (p \wedge q) \equiv \neg p \vee \neg q$

$p \wedge q \equiv \neg (\neg p \vee \neg q)$

$p \vee q \equiv \neg (\neg p \wedge \neg q)$

equality:

FOL includes other ways to make atomic sentences other than using a predicate and terms.

We can use the equality symbol to make the statements to the effect that 2 terms refer to the same object.

Father (John) = Henry.

objects referred to by Father (John) and the object referred to by Henry are the same.

Richard has atleast two brothers

$\exists x, y$ Brother $(x,$ Richard$) \wedge$ Brother $(y,$ Richard$)$
$\wedge \neg (x = y)$

Inferences in FOL

propositional Vs First order Inference.

- Inference in FOL is used to generate new sentences from existing sentences.

Inference in FOL (Def)

An expression α logically follows from a set s, if interpretation that satisfies 's' also satisfies α.

FR-SBIT

- The function of logical inference is to produce new sentence that logically follow a given set of FOL sentence.

~~Universal~~ Inference rules for quantifiers:

~~Universal Instantiation~~ (UI)

our KB contains the standard folklonic axiom All greedy kings are evil

$\forall x \; king(x) \wedge Greedy(x) \Rightarrow Evil(x)$

Infer the following statements.

Existential Instantiation: rule of inference
Existential quantifier is slightly more complicated.

- Any sentence α, variable v, and constant symbol k, that does not appear elsewhere in the KB.

$$\frac{\exists v \ \alpha}{SUBST(\{v/k\}, \alpha)}$$

$\exists x \ Crown(x) \wedge OnHead(x, John)$

we can infer the sentence.

$\exists c_1 \ Crown(c_1) \wedge OnHead(x, c_1)$

Skolem constant:
It is a new constant that is substituted for a variable when eliminating an existential quantifier from a fact or a universal quantifier.

Skolemization: It is a transformation on FOL formula, which eliminates all existential quantifiers from a formula.

$\exists x \ P(x)$ meaning that there exists x and $P(x)$ is true.

if $P(a)$ then it should be true.

If the statement is
"There exists a student who speaks German"

$(\exists x\ \text{student}(x) \wedge \text{speaksGerman}(x))$

Then existential instantiation allows to conclude that, "Jerry speaks German"

$(\text{student}(\text{Jerry}) \wedge \text{speaksGerman}(\text{Jerry}))$

- Universal Instantiation can be applied many times to produce many different consequences.
- Existential instantiation can be applied only once, and then the existentially quantified sentence can be discarded.
- We have added the sentence

  $\text{kill}(\text{Murderer}, \text{victim})$ we no longer need the sentence

  $\exists x\ \text{kill}(x, \text{victim})$

The above is not logically equivalent to the old, but it can be shown to be inferentially equivalent in the sense that it is satisfiable exactly when the original KB is satisfiable.

Name :
Year & Sem
Subject : AI
H.T. No.
Date
Signature of Invigilator

Unification and lifting:

Unification is a process that finds a substitution for variables in logical expressions.

It is used to determine if two expressions can be made identical.

Unification is a key component in logic programming and theorem proving

A first-order inference rule:

All Greedy kings are evils

$\forall x \; Greedy(x) \wedge king(x) \Rightarrow evil(x)$

The infer will give the conclusion

The inference process can be captured on a single inference rule that we call Generalized modus ponens

Atomic sentences P_i, P_i' and Q

$SUBST(\theta, P_i') = SUBST(\theta, P_i)$ for all i

$$\frac{P_1', P_2' \ldots P_n', (P_1 \wedge P_2 \ldots P_n \Rightarrow Q)}{SUBST(\theta, Q)}$$

There are n+1 premises to this rule.

n atomic sentences P_i'

and one implication.

The conclusion is the θ to the consequent q.

P_1' is King(John) P_1 King(x)

P_2' is Greedy(y) P_2 is Greedy(x)

θ is {x/John, y/John} q is Evil(x)

SUBST(θ, q) is Evil(John)

SUBST(θ, P_1') $\wedge$ ... SUBST(θ, P_n')

implication $P_1 \wedge P_2 \wedge \ldots \wedge P_n \Rightarrow q$ we can infer

SUBST(θ, P_1) $\wedge$ ... SUBST(θ, P_n) $\Rightarrow$ SUBST(θ, q)

θ is GMP

unification is an algorithm for determining the substitutions needed to make two FOL expressions match (predicate calculus).

- To apply rules of inference, an inference system must be able to determine when two expressions match, hence the unification algorithm is used.
- The UNIFY algorithm is used for unification which takes two atomic sentences and returns a unifier for those sentences.
- unification is a key component of all first order inference algorithms.

UNIFY (P, Q) = θ

where SUBST (θ, P) = SUBST (θ, Q)

where P and Q are two atomic sentences
θ is the substitution.
θ is the unifier value.
atom or atomic sentences.

King(John) ∧ Greedy(John) ⇒ Evil(John)
King(Richard) ∧ Greedy(Richard) ⇒ Evil(Richard)
King(Father(John)) ∧ Greedy(Father(John)) ⇒ Evil(Father(John))

Universal Instantiation : It is a rule of inference

Term is a logical expression that refers to an object.

A term with no variables called a ground term.

The rule of UI says that we can infer any sentence obtained by substituting a ground term

let SUBST(θ, α) denote the result of applying the substitution θ to the sentence α

Then rule is written as

$$\frac{\forall v\ \alpha}{SUBST(\{v/g\}, \alpha)}$$

for variable 'v' and ground term 'g'

The above 3 statements are substituted with {x/John}, {x/Richard}, and {x/Father(John)}

Following are some basic condi for unification.

- predicate symbol must be same, atom or expression with different predicate symbol can never be unified.
- no. of arguments in both expressions must be identical.
- unification will fail if there are 2 all or variables present in the same expression

UNIFY (knows (John, x), knows (John, Jane)) = {x/Jane}

~~UNIFY~~ (

performing unifications b/w 2 expressions the predicate symbol is same in both expression or in two cases.

- In each case 2 arguments are there so unification is possible.

x is replaced with Jane.

So both expressions are all or.

knows (John, Jane), knows (John, Jane).

UNIFY (knows (John, x), knows (y, Bill)) {x/Bill, y/John}

UNIFY (knows (John, x), knows (y, Mother(y)) {x/Mother(John), y/John}

θ = {x/Mother(John), y/John}

unify (knows(John, x), knows(x, elizabeth)) :
where we are having same variable the unification is not possible.

To apply unification we have to change the variable x with y.

unify (knows(John, x), knows(y, elizabeth))
{x/elizabeth, y/John}

The problem avoiding is called as standardizing apart.

- problem
  - Every one who loves all animals is loved by someone
  - Anyone who kills an animal is loved by no one.
  - Jack loves all animals
  - Either Jack or curiosity killed the cat, who is named Tuna.
  - Did curiosity kill the cat?